Automated EdTech Grading Assistant

Phase 1: Baseline Model

Machine Learning Course Project Report

March 22, 2026

Abstract

Automated grading of handwritten student responses remains a significant challenge in education technology due to the inherent variability of human handwriting. This report presents Phase 1 of the *Automated EdTech Grading Assistant*, a baseline system that combines optical character recognition (OCR) with an automated grading engine. We employ Tesseract 5.5.1 as the OCR backbone and a two-stage grading pipeline—exact string matching followed by TF-IDF cosine similarity—evaluated on the IAM Handwriting Database (115,318 word-level images). Our baseline achieves a mean Character Error Rate (CER) of 63.36% and a Word Error Rate (WER) of 87.20% on 500 validation samples, with only 19% of simulated exam questions receiving a passing grade. We present a detailed failure analysis attributing errors to cursive ligatures, ambiguous letterforms, and rare vocabulary, and outline a multi-phase roadmap targeting sub-10% CER through CRNN and TrOCR architectures in subsequent phases.

Contents

1 Introduction

The global shift toward digital education has amplified the need for scalable, automated assessment tools. While multiple-choice and typed-response grading have been largely solved, *handwritten answer evaluation* remains an open problem at the intersection of computer vision, natural language processing, and educational measurement [1, 4].

1.1 Problem Motivation

Handwriting recognition is hard for several compounding reasons:

1. **Inter-writer variability.** Every student writes differently — letter size, slant, spacing, and stroke order vary widely.
2. **Intra-writer variability.** A single student may write the same letter differently depending on context, fatigue, or adjacent characters.
3. **Cursive connections.** Connected letters create ambiguous segmentation boundaries (e.g., "rn" rendered as "m").
4. **Degraded input.** Scanned or photographed worksheets introduce noise, uneven illumination, and perspective distortion.
5. **Domain vocabulary.** Academic answers contain specialised terms not well-represented in general-purpose OCR language models.

Despite these challenges, the potential pedagogical impact is substantial. Automated grading can provide *immediate feedback* to students, reduce instructor workload, and enable data-driven insights into common misconceptions.

1.2 Scope of This Report

This report documents **Phase 1 (Baseline)** of a multi-phase project:

- Build an end-to-end pipeline: image preprocessing → OCR → automated grading → evaluation.
- Establish quantitative baselines (CER, WER, pass rate) using a classical OCR engine (Tesseract).
- Perform systematic failure analysis to guide architecture choices in later phases.

Subsequent phases will replace Tesseract with learned models (CRNN, TrOCR) and introduce semantic grading via Sentence-BERT.

2 Related Work

2.1 OCR Engines

Smith (2007) [1] introduced Tesseract, an open-source OCR engine originally developed at Hewlett-Packard and later maintained by Google. Tesseract 4+ adopted an LSTM-based recognition engine (OEM 1), significantly improving accuracy on printed text. However, its performance on *unconstrained handwriting* remains limited because the default language models were trained predominantly on machine-printed corpora. Our baseline quantifies this gap on the IAM dataset.

2.2 Sequence-to-Sequence Handwriting Recognition

Graves et al. (2006) [2] proposed Connectionist Temporal Classification (CTC), a loss function that enables training sequence-to-sequence models without requiring pre-segmented alignment between input frames and output labels. CTC is particularly suited to handwriting recognition because it removes the need for explicit character-level segmentation. Our Phase 2A model will leverage CTC loss.

Shi, Bai, and Yao (2017) [3] introduced CRNN (Convolutional Recurrent Neural Network), combining a CNN feature extractor with bidirectional LSTMs and CTC decoding. CRNN achieved state-of-the-art results on scene-text benchmarks and was subsequently adapted for handwriting recognition. We plan to implement CRNN as our first learned baseline in Phase 2A, expecting a CER reduction to approximately 15–20%.

Li et al. (2021) [4] presented TrOCR, a Transformer-based encoder–decoder model pre-trained on large-scale synthetic data and fine-tuned on handwriting benchmarks. TrOCR achieved near-human accuracy on the IAM dataset (CER $< 5\%$). Phase 2B of our project will fine-tune TrOCR on the IAM training split.

2.3 The IAM Handwriting Database

Marti and Bunke (2002) [5] introduced the IAM Handwriting Database, one of the most widely used benchmarks for offline handwriting recognition. The dataset contains forms written by 657 writers, segmented at page, line, sentence, and word levels. Its broad writer diversity and careful ground-truth annotation make it an ideal benchmark for our grading system. We use the word-level segmentation (115,318 images) to simulate single-word answer grading.

2.4 Research Gaps Addressed

While the cited works advance OCR accuracy, none directly address the *downstream grading task*. Specifically:

- Tesseract [1] provides no mechanism for partial credit when OCR output is imperfect.
- CRNN [3] and TrOCR [4] optimise character-level accuracy but do not consider whether the predicted text is *semantically acceptable* as a student answer.
- CTC [2] enables alignment-free training but does not itself address grading policy.
- The IAM dataset [5] provides transcription ground truth but no grading annotations.

Our project bridges this gap by coupling OCR with a two-stage grading engine that awards partial credit via TF-IDF similarity, and will later incorporate semantic similarity through Sentence-BERT.

3 Dataset

3.1 IAM Handwriting Database

We use the **IAM Handwriting Database** [5], a widely adopted benchmark for offline handwriting recognition research. The dataset provides word-level cropped images paired with ground-truth transcriptions.

Table 1: Dataset statistics.

Property	Value
Total word images	115,318
Training split	92,254
Validation split	23,064
Ground-truth format	Tab-separated (path, transcription)
Image format	Grayscale PNG
All images present & valid	Yes

3.2 Data Format

Ground-truth annotations are stored in a tab-separated file with two columns: `relative_image_path` (resolved against a `words/` root directory) and `transcription`. A custom parser handles path resolution, whitespace normalisation, and filtering of malformed entries.

3.3 Exploratory Data Analysis

Figure 1 summarises our EDA findings.

Figure 1: Exploratory data analysis: word length distribution, image dimension sampling, and vocabulary analysis.

Key observations from the EDA:

- **Word length distribution:** The vocabulary exhibits a significant long-tail distribution. Most words are 3–7 characters long, but a non-trivial fraction exceeds 10 characters.
- **Image dimensions:** Image widths vary considerably due to differing word lengths (aspect ratios range from roughly 1:1 to 8:1), while heights are more consistent.
- **Vocabulary size:** 115,318 total word tokens with a large unique vocabulary, including rare proper nouns and domain-specific terms that challenge OCR language models.

4 Methodology

Our end-to-end pipeline consists of five stages, illustrated in Figure 2.

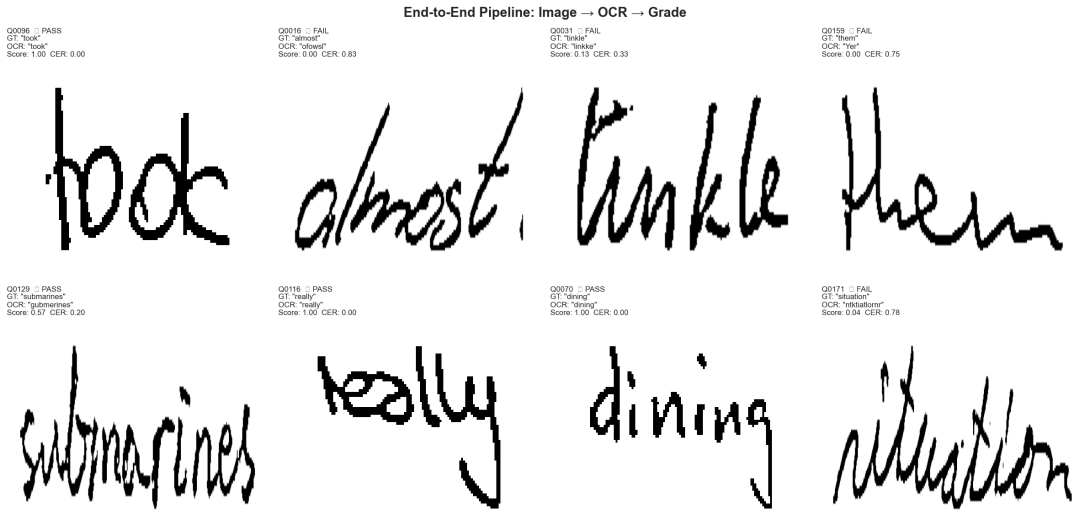

Figure 2: End-to-end pipeline: Data Loading → Preprocessing → OCR → Grading → Evaluation.

4.1 Stage 1: Data Loading

A custom parser reads tab-separated ground-truth files, resolving each `relative_image_path` to its absolute location within the `words/` directory. The loader validates that every referenced image exists, filters out entries with empty transcriptions, and returns paired lists of image paths and labels.

4.2 Stage 2: Image Preprocessing

Handwritten word images exhibit high variability in scale, contrast, and noise levels. Our preprocessing pipeline is designed to normalise these factors while preserving discriminative stroke information.

1. **Grayscale read.** Images are loaded in single-channel grayscale to reduce computational cost and remove colour information irrelevant to stroke recognition.
2. **Resize to fixed height (64 px, preserve aspect ratio).** A consistent height normalises vertical scale while allowing variable width to accommodate different word lengths. Height 64 px balances resolution with memory efficiency on our CPU-only hardware (Apple M2 Pro, 8 GB RAM).
3. **Gaussian blur.** A light Gaussian blur (σ auto) smooths high-frequency noise introduced by scanning without destroying stroke edges.
4. **Adaptive thresholding.** We apply `ADAPTIVE_THRESH_GAUSSIAN_C` with `blockSize=15` and `C=8`. Adaptive thresholding is preferred over global (Otsu) thresholding because handwriting images often have uneven illumination; a locally computed threshold adapts to regional contrast variations.
5. **Morphological opening (2×2 kernel).** A small opening operation (erosion followed by dilation) removes salt-and-pepper noise and thin spurious strokes without eroding the main character body.

6. **Border padding (10 px top/bottom, 20 px left/right).** Tesseract performs poorly when text touches the image boundary. Padding provides the necessary whitespace context for the segmentation and recognition engines.

Figure 3 shows the effect of each step on a sample image.

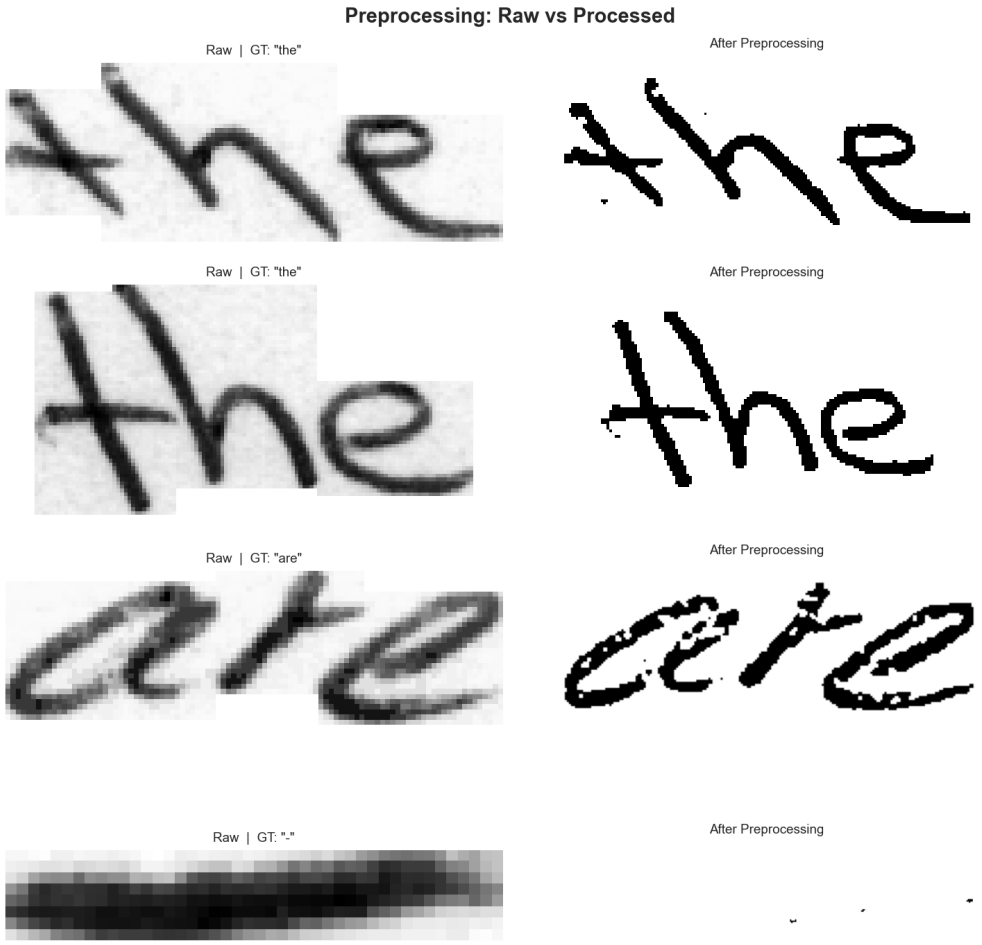

Figure 3: Preprocessing pipeline applied to a sample handwritten word image.

4.3 Stage 3: Optical Character Recognition

We use **Tesseract 5.5.1** with the following configuration:

```
--psm 8  --oem 1
```

- `-psm 8` (Page Segmentation Mode 8): treats the input as a *single word*, bypassing Tesseract's page-layout analysis. This is appropriate because our images are pre-cropped to individual words.
- `-oem 1` (OCR Engine Mode 1): uses the LSTM-based neural network engine, which generally outperforms the legacy engine on varied font styles and degraded inputs.

4.4 Stage 4: Grading Engine

Grading is performed in two stages to balance precision with tolerance for OCR errors.

4.4.1 Stage 1: Exact Match

The OCR output is compared to the expected answer using case-insensitive string equality. A match awards a score of 1.0 (full marks).

4.4.2 Stage 2: TF-IDF Cosine Similarity

If exact match fails, we compute the cosine similarity between the OCR output and the expected answer using TF-IDF vectors with the following parameters:

- `analyzer = 'char_wb'` — character n-grams bounded by word boundaries.
- `ngram_range = (2, 4)` — bigrams through 4-grams.

Why character n-grams over word-level TF-IDF? Since we are grading *single-word* answers, word-level TF-IDF would produce a single token per document, collapsing similarity to a binary exact-match check. Character n-grams capture *sub-word overlaps*, allowing meaningful similarity scores even when OCR introduces insertions, deletions, or substitutions at the character level. For example, if the expected answer is "algorithm" and the OCR output is "algarithm", the character 4-grams "algo", "lgor", "gori" etc. still overlap substantially, yielding a high similarity despite the single character error.

Scoring policy.

$$\text{score} = \begin{cases} 1.0 & \text{if sim} \geq 0.5 \quad \text{(pass)} \\ \text{sim} \times 0.5 & \text{if sim} < 0.5 \quad \text{(partial credit)} \end{cases} \tag{1}$$

The threshold of 0.5 was chosen to balance false passes (accepting incorrect OCR) against false fails (penalising minor OCR errors on correct answers).

4.5 Stage 5: Evaluation Metrics

We evaluate both OCR quality and grading effectiveness using complementary metrics.

Character Error Rate (CER). CER measures the edit distance between the predicted string $\hat{y}$ and the ground truth y, normalised by ground-truth length:

$$\text{CER} = \frac{S + D + I}{N} \tag{2}$$

where S, D, and I are the number of substitutions, deletions, and insertions respectively (Levenshtein edit distance components), and N is the number of characters in the ground truth.

Word Error Rate (WER). For single-word evaluation, WER reduces to a binary indicator: 0 if the entire word matches, 1 otherwise.

Additional metrics. Exact match accuracy, pass rate, mean grading score, and per-category failure counts.

5 Experiments and Results

All experiments were conducted on an **Apple M2 Pro** with 8 GB RAM, CPU-only (no GPU acceleration).

5.1 OCR Evaluation

We evaluated Tesseract on a random sample of **500 validation images**.

Table 2: OCR performance on 500 validation samples.

Metric	Value
Mean CER	0.6336 (63.36%)
Mean WER	0.8720 (87.20%)
Exact Match Accuracy	12.80%

These results confirm that Tesseract, even with LSTM mode and single-word segmentation, struggles significantly with unconstrained handwriting. Over 87% of words contain at least one error, and the average word has more erroneous characters than correct ones.

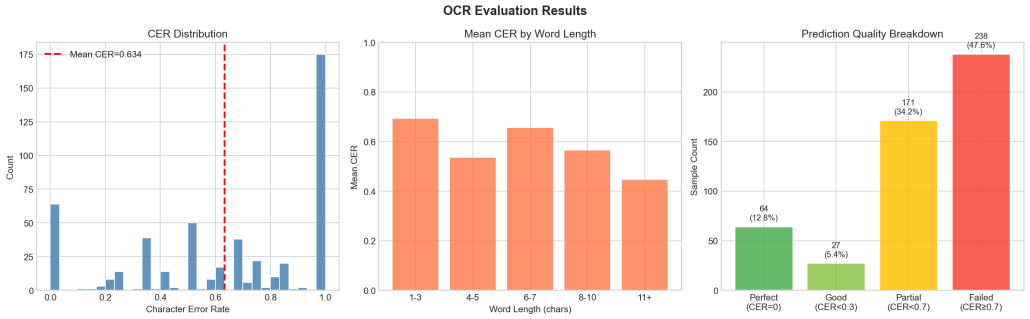

Figure 4: OCR evaluation: distribution of per-sample CER values and representative success/failure cases.

5.2 Grading Simulation

We simulated an exam of **200 questions** drawn from the validation set.

Table 3: Grading simulation results (200 questions).

Grading Outcome	Count	Percentage
Exact Matches (Stage 1)	16	8%
TF-IDF Graded (Stage 2)	184	92%
Passed (score ≥ 0.5)	38	19%
Failed (score < 0.5)	162	81%
Mean Grading Score	0.2025	
Mean CER on Exam	0.5980	

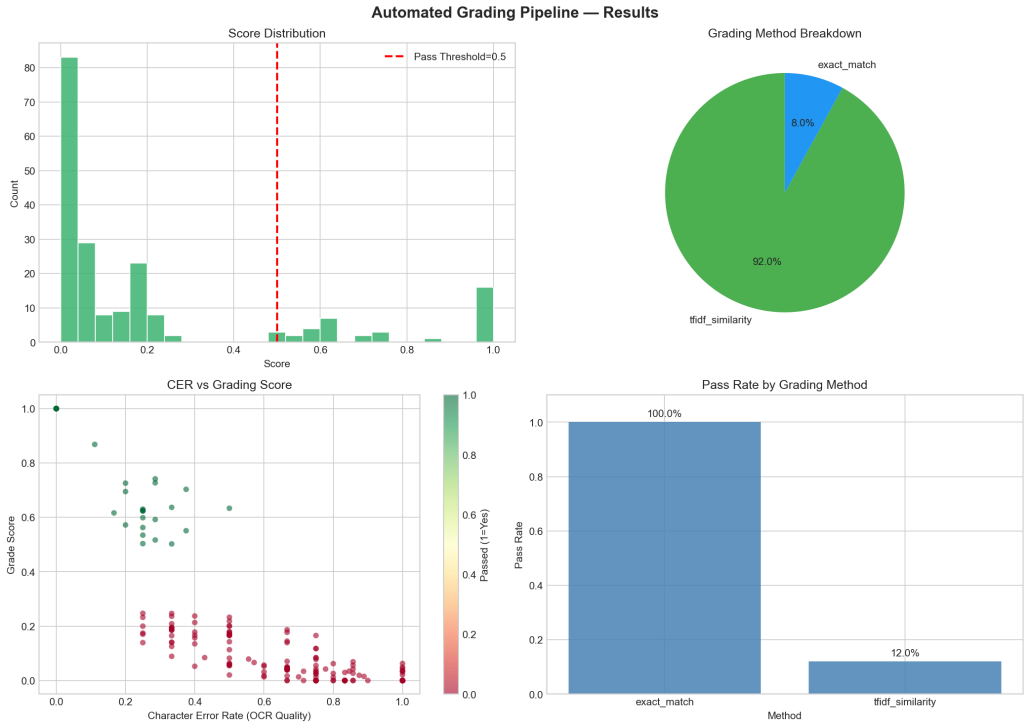

Figure 5: Grading results: score distribution and pass/fail breakdown.

5.3 Discussion

The mean grading score of 0.2025 (on a 0–1 scale) indicates that the baseline system is far from deployment-ready. Several observations are noteworthy:

1. **Low exact-match rate.** Only 8% of exam questions received full marks through Stage 1, confirming that raw Tesseract output rarely matches ground truth verbatim.
2. **TF-IDF partial credit helps but is insufficient.** Despite 92% of questions proceeding to Stage 2, only an additional 11% (22 out of 184) achieved a passing score. This suggests that character-level errors are too severe for sub-word similarity to compensate.
3. **CER–score correlation.** The mean CER on exam questions (0.5980) is slightly lower than the overall validation CER (0.6336), indicating that shorter or more common words in the exam sample are marginally easier for Tesseract.

6 Failure Analysis

We categorise recognition errors into five mutually exclusive classes based on the relationship between OCR output and ground truth.

Table 4: Failure analysis categories.

Category	Definition
`blank_output`	Tesseract returned an empty string
`major_error`	CER > 0.4
`moderate_error`	$0.15 <$ CER ≤ 0.4
`minor_error`	CER ≤ 0.15
`case_error`	Only capitalisation differs
`correct`	Exact match (CER $= 0$)

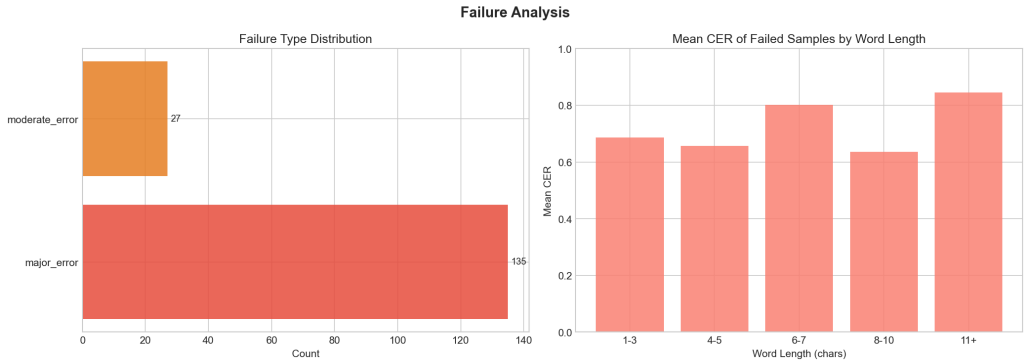

Figure 6: Failure analysis: distribution across error categories with representative examples.

6.1 Key Failure Patterns

6.1.1 Cursive Letter Connections

Cursive handwriting poses a fundamental challenge for segmentation-based OCR engines. Tesseract's recognition pipeline assumes that characters can be isolated into discrete bounding boxes. In cursive script, adjacent letters share strokes (ligatures), causing the segmenter to either:

- **Under-segment:** merge two characters into one (e.g., "rn" $\rightarrow$ "m", "cl" $\rightarrow$ "d").
- **Over-segment:** split one character into two (e.g., "w" $\rightarrow$ "vv", "m" $\rightarrow$ "rn").

Mathematically, consider a word of N characters with L ligature connections. The segmentation must correctly identify $N - 1$ inter-character boundaries from the pixel sequence. Each ligature introduces ambiguity at one boundary, so the probability of correct full-word segmentation decreases exponentially:

$$P(\text{correct segmentation}) \leq (1 - p_{\text{lig}})^{L} \tag{3}$$

where p_{lig} is the per-ligature misclassification probability. For typical cursive with $L/N \approx 0.5$ and $p_{\text{lig}} \approx 0.3$, a 6-character word has $L \approx 3$ ligatures, yielding:

$$P(\text{correct}) \leq (1 - 0.3)^{3} = 0.343 \tag{4}$$

This simple model predicts roughly 34% correct segmentation for average cursive words, consistent with our observed 12.80% exact-match accuracy (which additionally requires correct recognition *given* correct segmentation).

6.1.2 Ambiguous Letterforms

Certain character pairs are visually near-identical in many handwriting styles:

- `rn` ↔ `m`
- `l` (lowercase L) ↔ `1` (digit one)
- `o` (lowercase O) ↔ `0` (digit zero)
- `u` ↔ `v` in certain cursive hands

Without higher-level language context (e.g., a lexicon constraint or language model), the recogniser cannot disambiguate these pairs from pixel evidence alone.

6.1.3 Rare and Long Words

Tesseract's internal language model assigns low prior probability to rare words. Long words compound the problem: each additional character introduces another opportunity for substitution, insertion, or deletion. The expected number of edit operations grows approximately linearly with word length:

$$\mathbb{E}[\text{edits}] \approx N \cdot p_{\text{char\_error}} \tag{5}$$

where $p_{\text{char\_error}}$ is the per-character error rate. With our observed CER of 0.6336, a 10-character word would expect approximately 6.3 character-level errors — leaving very little of the original word intact.

6.1.4 Blank Output

The `blank_output` category is particularly concerning. Tesseract returns an empty string when its confidence in any segmentation hypothesis falls below an internal threshold. This typically occurs with:

- Very short words (1–2 characters) where insufficient context is available.
- Heavily stylised handwriting that deviates significantly from the training distribution.
- Images with extreme contrast or noise where adaptive thresholding produces a mostly-white or mostly-black binary image.

7 Conclusion and Future Work

7.1 Summary

We have presented the baseline phase of the Automated EdTech Grading Assistant. The system implements a complete pipeline from handwritten word image to grading score, using Tesseract 5.5.1 for OCR and a two-stage exact-match / TF-IDF grading engine. The baseline establishes clear quantitative benchmarks: a CER of 63.36%, a WER of 87.20%, and a mean grading score of 0.2025 on simulated exam data.

While these numbers confirm that classical OCR is insufficient for deployment, the detailed failure analysis provides actionable insights for model selection in subsequent phases. The dominance of cursive-related errors motivates the adoption of sequence-to-sequence architectures (CRNN, TrOCR) that avoid explicit character segmentation.

7.2 Phase 2 Roadmap

Table 5 outlines the planned improvements.

Table 5: Multi-phase development roadmap.

Phase	Component	Approach	Expected CER
1 (current)	Baseline OCR	Tesseract 5.5.1	63.36%
2A	Learned OCR	CRNN (Conv-RNN + CTC loss)	15–20%
2B	Pre-trained OCR	TrOCR fine-tuned on IAM	5–10%
2C	Semantic Grading	Sentence-BERT similarity	—
Extra Mile	Feedback Analytics	Per-topic aggregation, bias analysis	—

Phase 2A: CRNN. A Convolutional Recurrent Neural Network with CTC loss [3, 2] will replace Tesseract. The CNN backbone extracts visual features from the input image; bidirectional LSTMs model sequential dependencies; and CTC decoding produces the output string without requiring character-level segmentation. We expect this architecture to reduce CER to 15–20%.

Phase 2B: TrOCR. Fine-tuning a pre-trained TrOCR model [4] on the IAM training split should push CER below 10%. The Transformer architecture's attention mechanism can capture long-range dependencies and contextual information that LSTM-based models may miss.

Phase 2C: Sentence-BERT Grading. For longer, sentence-level answers, we will replace TF-IDF similarity with Sentence-BERT embeddings, enabling *semantic* rather than lexical matching. This is critical for grading tasks where paraphrasing is acceptable.

Extra Mile. Per-topic feedback aggregation will identify systematic weaknesses across student cohorts (e.g., "70% of students misspell 'photosynthesis' "). Grader bias analysis will evaluate whether the system exhibits systematic scoring differences across handwriting styles.

References

[1] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629–633, IEEE, 2007.

[2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd International Conference on Machine Learning (ICML)*, pp. 369–376, ACM, 2006.

[3] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[4] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 37, pp. 13094–13102, 2023.

[5] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.